

Énoncé : Le Problème du Sac à Dos 0/1 (Version Objets)

Contexte

Vous disposez d'un sac à dos d'une capacité maximale donnée (un poids) et d'une liste d'objets. Chaque objet possède une **valeur** (le gain s'il est emporté) et un **poids**.

Le problème du "Sac à Dos 0/1" stipule que vous ne pouvez pas fractionner les objets : soit vous prenez l'objet en entier (1), soit vous le laissez (0).

Objectif

L'objectif est de maximiser la valeur totale des objets emportés dans le sac sans jamais dépasser la capacité maximale autorisée.

Consignes de codage

Vous devez écrire une fonction `knapsack(capacite, objets)` en Python.

- **capacite** : Un entier représentant le poids maximal que le sac peut supporter.
- **objets** : Une liste de dictionnaires. Chaque dictionnaire représente un objet et suit la structure suivante : `{"valeur": int, "poids": int}`.
- **Valeur de retour** : La fonction doit renvoyer un **entier** représentant la valeur totale maximale optimale.

⚠ Attention aux pièges d'implémentation :

- **Stratégies gloutonnes** : Choisir les objets selon le meilleur ratio valeur/poids ou selon le poids le plus léger ne donne pas toujours la solution optimale (voir cas de tests 5 et 6). Votre algorithme doit trouver l'optimum global.
- **Complexité et Performance** : Une approche récursive naïve (force brute) aura une complexité exponentielle en $O(2^n)$ et échouera sur les grands ensembles d'objets. Vous devez optimiser votre solution (par exemple, via la **programmation dynamique** ou la **mémoïsation**).

Exemples de comportement attendu

Exemple 1 : Cas classique (et piège glouton)

```
objets = [  
    {"valeur": 65, "poids": 12}, # ratio = 5.41  
    {"valeur": 50, "poids": 10}, # ratio = 5.0  
    {"valeur": 50, "poids": 10}  # ratio = 5.0  
]  
# Si la capacité est de 20 :  
# Un algo glouton prendrait le premier objet (poids 12), il ne reste que 8 de  
# place (impossible de mettre les autres). Valeur = 65.  
# L'optimum est de prendre les deux derniers objets (poids 10 + 10 = 20). Valeur =  
# 100.  
  
print(knapsack(20, objets))
```

```
# Sortie attendue : 100
```

Exemple 2 : Objets identiques

```
objets = [{"valeur": 20, "poids": 5}, {"valeur": 20, "poids": 5}, {"valeur": 20, "poids": 5}]
print(knapsack(11, objets))
# Sortie attendue : 40 (On peut en prendre exactement 2 sur les 3)
```

Prototype de la fonction

```
def knapsack(capacite: int, objets: list[dict]) -> int:
    """
    Calcule la valeur maximale possible sans dépasser la capacité du sac.

    :param capacite: Poids maximal supporté par le sac (int)
    :param objets: Liste de dictionnaires, ex: [{"valeur": 10, "poids": 2}]
    :return: Valeur maximale totale (int)
    """
    # Votre code ici
    pass
```

tests

```
print("Démarrage des tests pour le Sac à Dos 0/1 (Format Objets)...")

# Test 1 : Sac vide ou capacité nulle
print("Test 1...", knapsack(0, [{"valeur": 10, "poids": 1}, {"valeur": 20, "poids": 2}]))
assert knapsack(0, [{"valeur": 10, "poids": 1}, {"valeur": 20, "poids": 2}]) == 0, "Échec Test 1 : Capacité 0 doit renvoyer 0"

# Test 2 : Un seul objet disponible et il rentre
print("Test 2...", knapsack(10, [{"valeur": 100, "poids": 5}]))
assert knapsack(10, [{"valeur": 100, "poids": 5}]) == 100, "Échec Test 2 : L'objet doit être pris"

# Test 3 : Un seul objet disponible mais trop lourd
print("Test 3...", knapsack(10, [{"valeur": 100, "poids": 15}]))
assert knapsack(10, [{"valeur": 100, "poids": 15}]) == 0, "Échec Test 3 : Objet trop lourd, valeur 0"
```

```
# Test 4 : Tous les objets rentrent
objets_tous = [{"valeur": 10, "poids": 1}, {"valeur": 20, "poids": 2}, {"valeur": 30, "poids": 3}]
print("Test 4...", knapsack(10, objets_tous))
assert knapsack(10, objets_tous) == 60, "Échec Test 4 : Tout rentre, on additionne tout"

# Test 5 : Le piège glouton classique (Meilleur ratio valeur/poids qui bloque l'optimum global)
objets_piege_ratio = [
    {"valeur": 65, "poids": 12}, # ratio 6 (meilleur)
    {"valeur": 50, "poids": 10}, # ratio 5
    {"valeur": 50, "poids": 10} # ratio 5
]
print("Test 5...", knapsack(20, objets_piege_ratio))
assert knapsack(20, objets_piege_ratio) == 100, "Échec Test 5 : Piège glouton de ratio ! L'optimum est 100 (50+50)"

# Test 6 : Le piège glouton par le poids (Prendre le plus léger d'abord échoue)
objets_piege_poids = [
    {"valeur": 10, "poids": 2}, # Plus léger
    {"valeur": 100, "poids": 5} # Plus lourd mais beaucoup plus rentable
]
print("Test 6...", knapsack(5, objets_piege_poids))
assert knapsack(5, objets_piege_poids) == 100, "Échec Test 6 : Piège glouton sur le poids léger"

# Test 7 : Objets identiques (Choix multiples possibles)
objets_identiques = [{"valeur": 20, "poids": 5}, {"valeur": 20, "poids": 5}, {"valeur": 20, "poids": 5}]
print("Test 7...", knapsack(11, objets_identiques))
assert knapsack(11, objets_identiques) == 40, "Échec Test 7 : Doit prendre exactement 2 objets sur 3"

# Test 8 : Performance (Grand nombre d'objets - Risque d'explosion 2^n sans DP)
objets_grand = [
    {"valeur": 10, "poids": 1}, {"valeur": 20, "poids": 2}, {"valeur": 30, "poids": 3},
    {"valeur": 40, "poids": 4}, {"valeur": 50, "poids": 5}, {"valeur": 60, "poids": 6},
    {"valeur": 70, "poids": 7}, {"valeur": 80, "poids": 8}, {"valeur": 90, "poids": 9},
    {"valeur": 100, "poids": 10}, {"valeur": 110, "poids": 11}, {"valeur": 120, "poids": 12},
    {"valeur": 130, "poids": 13}, {"valeur": 140, "poids": 14}, {"valeur": 150, "poids": 15},
    {"valeur": 160, "poids": 16}, {"valeur": 170, "poids": 17}, {"valeur": 180, "poids": 18},
    {"valeur": 190, "poids": 19}, {"valeur": 200, "poids": 20}
]
print("Test 8 (Test de performance)...", knapsack(50, objets_grand))
assert knapsack(50, objets_grand) == 500, "Échec Test 8 : Trop lent ou mauvaise valeur"
```

```
# Test 9 : Performance extrême avec grande capacité
objets_ext = [
    {"valeur": 5, "poids": 50}, {"valeur": 10, "poids": 100}, {"valeur": 15,
    "poids": 150},
    {"valeur": 20, "poids": 200}, {"valeur": 25, "poids": 250}, {"valeur": 30,
    "poids": 300}
]
print("Test 9 (Test de performance grande capacité)...", knapsack(1000,
objets_ext))
assert knapsack(1000, objets_ext) == 100, "Échec Test 9 : Échec sur une capacité
élevée"

print("\nTous les tests du Sac à Dos (Format Objets) sont passés avec succès !")
```